By Nitesh Singh

Job Ready Free Live Training To Become A

SOC

Analyst L1 For Free

Defronix Cyber Security → https://defronix.com

What is Hashing?

A **hash** is a one-way mathematical transformation of data into a fixed-length string.

Think of it like:

Input → Hash Function → Fixed Length Output

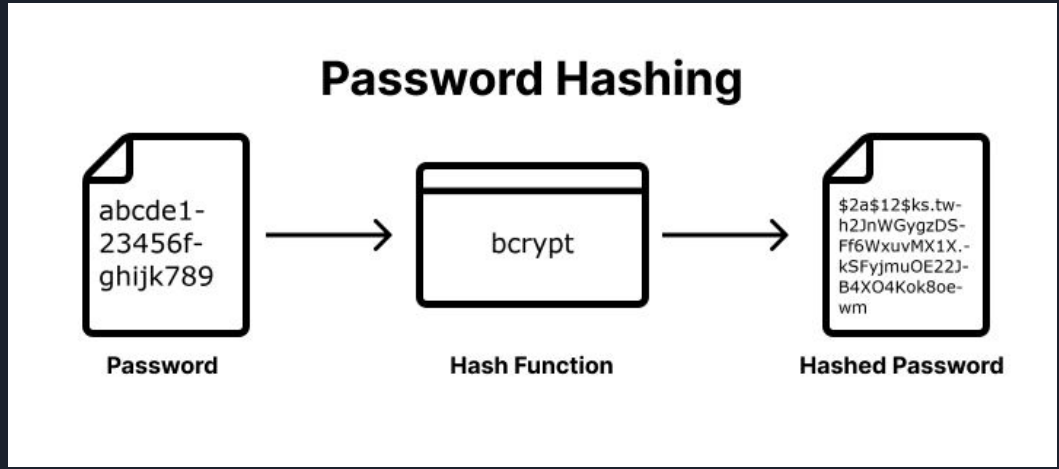
If password is:

password123

Using MD5, it becomes:

482c811da5d5b4bc6d497ffa98491e38

No matter how long input is → output length is fixed.



Properties of Hashing

- One-way (cannot reverse easily)
- Deterministic (same input → same output)
- Fixed output length
- Small input change → completely different output

Example:

password123
password124

Completely different hashes.

This is called the **Avalanche Effect**.

Common Hash Algorithms

Algorithm	Secure Today?	Notes
MD5	❌ No	Very weak
SHA1	❌ No	Broken
SHA256	✅ Yes	Strong
SHA512	✅ Yes	Strong
bcrypt	✅ Very Strong	Designed for passwords
Argon2	✅ Very Strong	Modern standard

Why Hash Passwords?

Imagine a website stores passwords in plain text:

username: nitesh
password: Nitesh@123

If database is hacked → attacker sees everything.

Instead, website stores:

username: nitesh
hash: 5e884898da...

Even if database leaks → attacker must crack hash first.

The Big Problem (Without Salting)

Hashing alone is NOT enough.

Because:

If two users use same password:

User1 → password123
User2 → password123

Both hashes will be identical.

Attacker instantly knows:

- They share same password.

Defronix Cyber Security → https://defronix.com

Rainbow Table Attack

Attackers create huge precomputed tables like:

Password	Hash
123456	e10adc3949ba...
password	5f4dcc3b5aa...
admin123	0192023a7bb...

If database hash matches one in table → password instantly cracked.

This is why MD5/SHA1 are dangerous.

What is Salting

Salt = Random value added to password before hashing.

Instead of hashing:

password123

System hashes:

password123 + random salt

User password:

password123

Salt generated:

x7K9pL2

System hashes:

password123x7K9pL2

Output:

a83bd981fa...

Another user with same password gets different salt → different hash.

Why Salting is Powerful

Even if 100 users use same password:

Without salt:
→ Same hash

With salt:
→ All hashes different

Attacker cannot use rainbaow tables effectively.

They must brute force each hash individually.

This makes cracking extremely expensive.

How Salting is Stored

Database stores:

Username
Salt
Hash

When user logs in:

1. System retrieves salt
2. Adds salt to entered password
3. Hashes it
4. Compares with stored hash

If match → login success

Hashing vs Encryption (Very Important Difference)

Feature	Hashing	Encryption
Reversible?	No	Yes
Used for passwords?	Yes	No
Needs key?	No	Yes
Purpose	Integrity	Confidentiality

Passwords should NEVER be encrypted.
They should be hashed + salted.

SOC Perspective: Why You Must Understand This

During incident response:

If database leaked,
You must assess:

- Was hashing used?
- Was salting used?
- Which algorithm?
- Was it bcrypt or just MD5?

If weak hashing → high risk of credential stuffing later.

Defronix Cyber Security → https://defronix.com

Real Attack Scenario

1. Company stores passwords in MD5 without salt
2. Database breached
3. Attacker cracks hashes in minutes
4. Users reuse same password on:
    - Email
    - VPN
    - Banking
5. Massive credential stuffing campaign begins

SOC sees:

- Login attempts from multiple IPs
- Account takeovers
- Impossible travel alerts

Root cause?
Weak hashing practice.

Advanced Concept: Pepper

Pepper = Secret value stored outside database (e.g., server config).

Instead of:

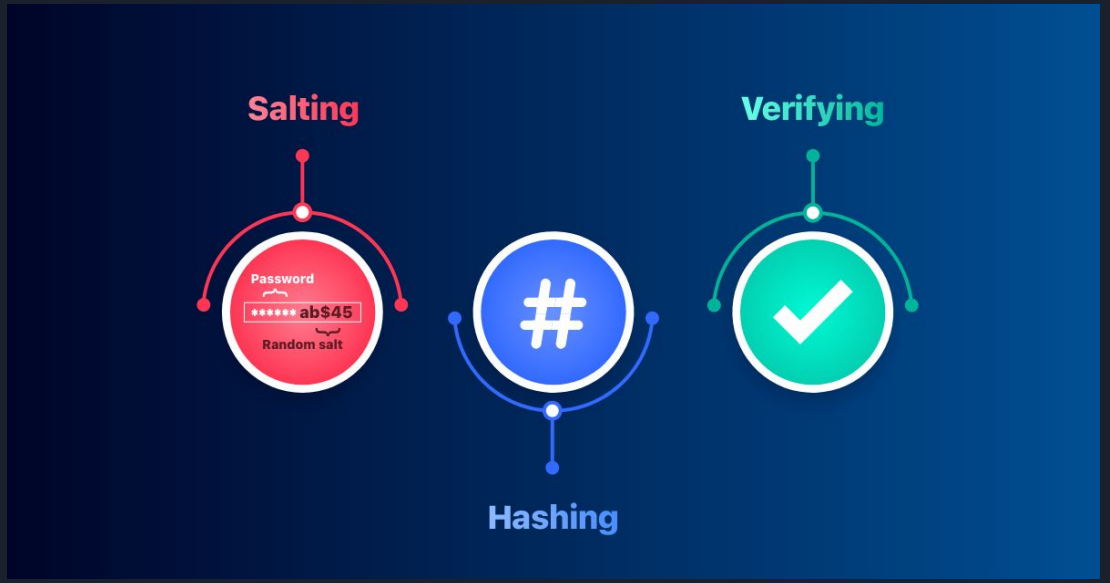
password + salt

It becomes:

password + salt + pepper

Even if database leaks → attacker doesn't know pepper.

Extra layer of protection.



What is Encryption?

Encryption is the process of converting readable data (plaintext) into unreadable data (ciphertext) using a **cryptographic key**, so that only authorized parties can read it.

Basic flow:

Plaintext → Encryption Algorithm + Key → Ciphertext
Ciphertext → Decryption Algorithm + Key → Plaintext

So encryption is **reversible** if you have the correct key.

Example

Original message:

BankTransfer = ₹50,000

Encrypted message (ciphertext):

A7F9C1B2E88DA91...

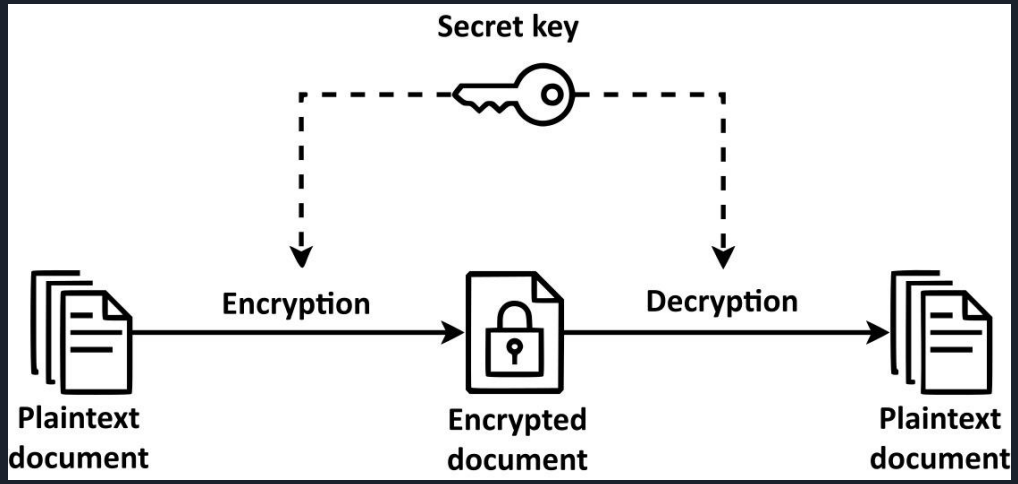
With the **correct key**, it can be decrypted back to the original message.

Why Encryption is Used

Encryption protects **data confidentiality**, meaning only authorized users can read the data.

Common uses:

- HTTPS website traffic
- VPN communication
- Disk encryption
- Secure messaging
- File encryption
- Database encryption
- Email encryption



Types of Encryption

Symmetric Encryption

Uses **one single key** for both encryption and decryption.

Example:

Plaintext + SecretKey → Ciphertext
Ciphertext + SecretKey → Plaintext

Example uses:

- Disk encryption
- VPN tunnels
- File encryption

Problem: **Key sharing must be secure.**

Popular algorithms:

Algorithm	Use
AES	Most common modern encryption
DES	Old and insecure
3DES	Legacy
ChaCha20	Modern secure encryption

Types of Encryption

Asymmetric Encryption (Public Key Cryptography)

Uses **two keys**:

- Public key (shared with everyone)
- Private key (kept secret)

Process:

Public Key → Encrypt
Private Key → Decrypt

Example uses:

- HTTPS websites
- SSL/TLS handshake
- Email encryption (PGP)

Popular algorithms:

Algorithm	Use
RSA	SSL/TLS
ECC	Modern secure communications
Diffie-Hellman	Key exchange

HTTPs Example : Encryption

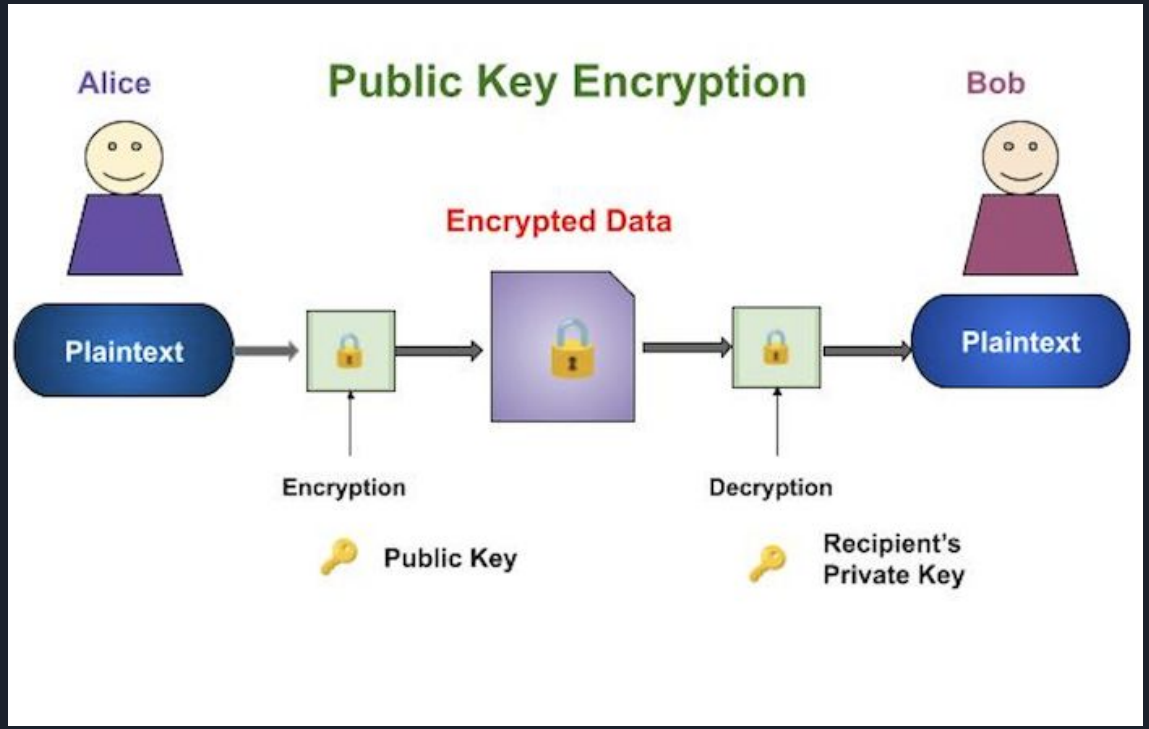
When you visit a website:

Browser → Server

Steps:

1. Browser gets **public key** from server
2. Browser encrypts session key
3. Server decrypts using **private key**
4. Secure communication begins

That's how **HTTPS works**.



Encryption vs Hashing

Feature	Encryption	Hashing
Reversible	Yes	No
Uses key	Yes	No
Purpose	Protect data confidentiality	Verify data integrity
Output length	Variable	Fixed
Example use	HTTPS traffic	Password storage
Can original data be retrieved	Yes	No

SOC Analyst Perspective

Understanding this helps in:

- Investigating data breaches
- Analyzing password dumps
- Understanding credential theft
- Investigating encrypted traffic
- Malware analysis

Example SOC question:

If attackers steal:

- encrypted database → harder to access
- hashed passwords → may crack offline